Compte rendu de TP — DNSSEC

Services Réseaux Avancés • BUT Réseaux & Télécommunications

Nous présentons les manipulations visibles dans la conversation du TP. Les captures intégrées montrent les commandes et résultats effectivement relevés. Lorsque la conversation ne permet pas de retrouver une valeur précise, nous le signalons.

I – Test de différents types de requêtes

A) Tests divers avec les outils en ligne

1 – Requête avec Google Dig

Nous avons interrogé un nom de site avec Google Dig et observé les sections de la réponse. L'adresse exacte retournée n'est pas lisible dans les éléments récupérés.

2 – Même requête avec Dig Web Interface en mode « no recursive »

Nous avons relancé la requête sans récursivité. La réponse ne donne pas nécessairement l'adresse finale : il faut poursuivre les requêtes auprès des serveurs indiqués ou interroger un résolveur récursif.

3 – Recherche de www.google.fr

La recherche a été effectuée, mais l'adresse IP exacte n'est pas assez lisible pour être reportée sans risque d'erreur.

4 – Informations sur uca.fr

La recherche des enregistrements et sous-domaines a été lancée. Les valeurs détaillées ne sont pas lisibles dans le partage, donc nous ne les reconstituons pas.

B) Tests avec dig sur la machine

1 – Résolution de google.com

Nous avons interrogé le résolveur configuré puis un serveur DNS précisé avec @. La capture Wireshark n'étant pas exploitable en détail, les adresses et champs exacts ne sont pas retranscrits.

```
dig google.com
dig @8.8.8.8 google.com
```

La première commande utilise le résolveur configuré sur la machine ; la seconde interroge explicitement 8.8.8.8.

2 – Nouvelle résolution de google.com

La requête a été relancée ; les réponses exactes ne sont pas disponibles pour confirmer si l'adresse était identique.

3 – Requête avec +trace

```
dig +trace google.com
```

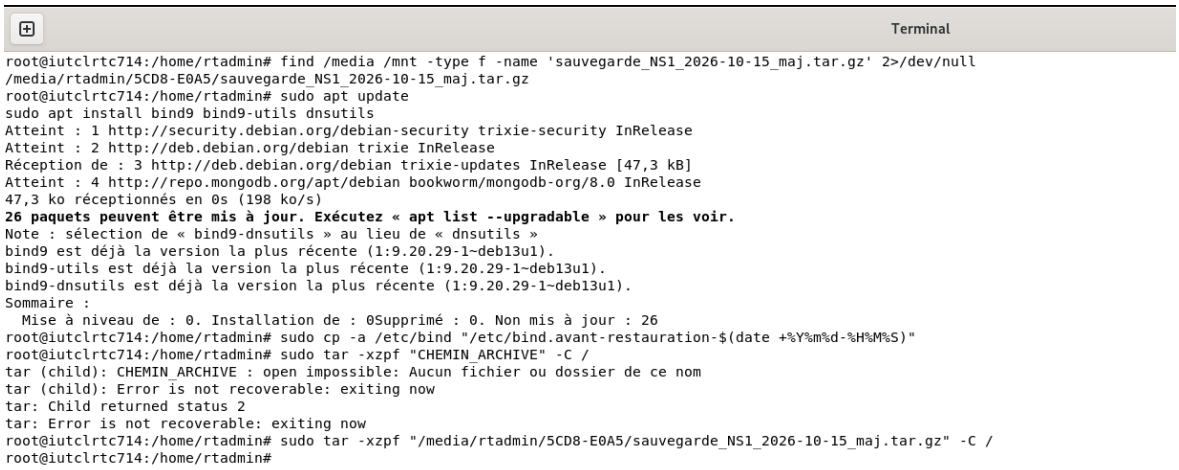
Cette option affiche les étapes de résolution depuis les serveurs racine. La sortie détaillée n'est pas récupérable dans le partage.

II – Installation d’un serveur DNS

1–2 – Installation et vérification de BIND9

Nous avons vérifié les paquets nécessaires sur la machine primaire. La capture montre que **bind9**, **bind9-utils** et **dnsutils** étaient déjà installés. Nous avons ensuite contrôlé le chargement de la zone et l’état du service ; BIND9 est indiqué comme actif.

```
sudo apt update
sudo apt install bind9 bind9-utils dnsutils
sudo systemctl status bind9 --no-pager
```



Capture 1 — Vérification des paquets sur NS1.

3 – Port utilisé par le service DNS

DNS utilise le port 53 en UDP pour la plupart des requêtes et en TCP notamment pour les transferts ou les réponses volumineuses. La capture ne permet pas de confirmer une vérification locale du port.

4 – Fichiers et zones BIND9

Les captures confirment le chargement de secu.rt et de 0.25.172.in-addr.arpa. La liste initiale des zones et le contenu du fichier de la zone racine « . » ne sont pas lisibles ; nous ne les détaillons donc pas.

5 – Commande rndc dumpdb

rndc dumpdb demande à BIND9 d’écrire le contenu de sa base en mémoire, notamment son cache, dans un fichier de dump. Le test demandé avec google.com et yahoo.com n’est pas confirmé par une sortie lisible dans la conversation.

6 – Traceroute vers les serveurs racine

Un traceroute affiche les routeurs traversés jusqu’à une destination. Les commandes et chemins observés ne sont pas lisibles dans les éléments récupérés.

III – Configuration d'un serveur primaire

A) Création d'une nouvelle zone

Nous avons configuré la zone directe **secu.rt** sur le serveur primaire. Le sujet demande les enregistrements NS, SOA, A et les alias ; la capture de vérification confirme les adresses utilisées pour les serveurs, le client et le serveur mail.

```
sudo named-checkzone secu.rt /etc/bind/db.secu
sudo named-checkconf -z
sudo systemctl restart bind9
sudo systemctl status bind9 --no-pager
```

Dans la configuration relevée, **ns1.secu.rt** utilise 172.25.0.74, **ns2.secu.rt** 172.25.0.75, **client.secu.rt** 172.25.0.63 et **mail.secu.rt** 172.25.0.77. La zone est chargée et les contrôles affichent **OK**.

```
root@iutclrtb834:/home/rtadmin/Téléchargements/Sauvegarde-TP-DNSSEC-2026-10-06/Sauvegarde-TP-DNSSEC# sudo named-checkconf
root@iutclrtb834:/home/rtadmin/Téléchargements/Sauvegarde-TP-DNSSEC-2026-10-06/Sauvegarde-TP-DNSSEC# sudo named-checkzone secu.rt /etc/bind/db.secu.signed
zone secu.rt/IN: loaded serial 2026101501 (DNSSEC signed)
OK
root@iutclrtb834:/home/rtadmin/Téléchargements/Sauvegarde-TP-DNSSEC-2026-10-06/Sauvegarde-TP-DNSSEC# sudo named-checkzone secu.rt /etc/bind/db.secu.signed
zone secu.rt/IN: loaded serial 2026101501 (DNSSEC signed)
OK
root@iutclrtb834:/home/rtadmin/Téléchargements/Sauvegarde-TP-DNSSEC-2026-10-06/Sauvegarde-TP-DNSSEC# sudo systemctl restart bind9
sudo systemctl status bind9 --no-pager
● named.service - BIND Domain Name Server
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
     Active: active (running) since Thu 2026-10-15 18:27:05 CEST; 12ms ago
 Invocation: 1dbc3a00de80409a9662f704a8932903
       Docs: man:named(8)
   Main PID: 6103 (named)
     Status: "running"
      Tasks: 50 (limit: 18618)
     Memory: 219.7M (peak: 220.7M)
        CPU: 50ms
     CGroup: /system.slice/named.service
             └─6103 /usr/sbin/named -f -u bind

oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './DNSKEY/IN': 2001:500:2f::f#53
oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './NS/IN': 2001:500:2f::f#53
oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './DNSKEY/IN': 2001:500:9f::42#53
oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './NS/IN': 2001:dc3::35#53
oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './DNSKEY/IN': 2001:dc3::35#53
oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './NS/IN': 2001:500:9f::42#53
oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './DNSKEY/IN': 2001:500:2d::d#53
oct. 15 18:27:05 iutclrtb834 named[6103]: network unreachable resolving './NS/IN': 2001:500:2d::d#53
oct. 15 18:27:05 iutclrtb834 named[6103]: managed-keys-zone: Key 20326 for zone . is now trusted (acceptance timer complete)
oct. 15 18:27:05 iutclrtb834 named[6103]: managed-keys-zone: Key 38696 for zone . is now trusted (acceptance timer complete)
root@iutclrtb834:/home/rtadmin/Téléchargements/Sauvegarde-TP-DNSSEC-2026-10-06/Sauvegarde-TP-DNSSEC# sudo nano /etc/bind/named.conf.local
```

Capture 2 — Zone signée vérifiée et service BIND9 actif.

```
root@iutclrtc714:/home/rtadmin# sudo named-checkzone secu.rt /etc/bind/db.secu
zone secu.rt/IN: loaded serial 2026101501
OK
root@iutclrtc714:/home/rtadmin# grep -E 'serial|^(ns1|ns2|client|mail)[[:space:]]+IN[[:space:]]+A' /etc/bind/db.secu
                        2026101501 ; serial YYYYMMDDnn
ns1     IN      A       172.25.0.74
ns2     IN      A       172.25.0.75
client  IN      A       172.25.0.63
mail    IN      A       172.25.0.77
root@iutclrtc714:/home/rtadmin# sudo dnssec-signzone -K /etc/bind/keys -3 A1B2C3D4 -o secu.rt -x -f /etc/bind/db.secu.nsec3 /etc/bind/db.secu
Verifying the zone using the following algorithms:
- RSASHA256
Zone fully signed:
Algorithm: RSASHA256: KSKs: 1 active, 0 stand-by, 0 revoked
                      ZSKs: 1 active, 0 present, 0 revoked
/etc/bind/db.secu.nsec3
root@iutclrtc714:/home/rtadmin# sudo named-checkzone secu.rt /etc/bind/db.secu.nsec3
zone secu.rt/IN: loaded serial 2026101501 (DNSSEC signed)
OK
root@iutclrtc714:/home/rtadmin# sudo named-checkconf -z
zone secu.rt/IN: loaded serial 2026101501 (DNSSEC signed)
zone 0.25.172.in-addr.arpa/IN: loaded serial 2026101501
root@iutclrtc714:/home/rtadmin# sudo systemctl restart bind9
sudo systemctl status bind9 --no-pager
● named.service - BIND Domain Name Server
     Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-20 14:08:43 CEST; 16ms ago
 Invocation: 59f1a40f7f344ab195ebae97015c44dd
       Docs: man:named(8)
   Main PID: 4076 (named)
     Status: "running"
      Tasks: 50 (limit: 18600)
     Memory: 215M (peak: 216.4M)
        CPU: 87ms
     CGroup: /system.slice/named.service
             └─4076 /usr/sbin/named -f -u bind

oct. 20 14:08:43 iutclrtc714 named[4076]: network unreachable resolving './DNSKEY/IN': 2001:500:9f::42#53
oct. 20 14:08:43 iutclrtc714 named[4076]: network unreachable resolving './NS/IN': 2001:500:9f::42#53
oct. 20 14:08:43 iutclrtc714 named[4076]: network unreachable resolving './DNSKEY/IN': 2001:500:a8::e#53
oct. 20 14:08:43 iutclrtc714 named[4076]: network unreachable resolving './NS/IN': 2001:500:a8::e#53
oct. 20 14:08:43 iutclrtc714 named[4076]: network unreachable resolving './DNSKEY/IN': 2001:500:2f::f#53
oct. 20 14:08:43 iutclrtc714 named[4076]: network unreachable resolving './NS/IN': 2001:500:2f::f#53
```

Capture 3 — Contrôle de la zone et adresses configurées sur NS1.

A.1–2 – Déclaration et contenu de la zone

Nous avons chargé la zone directe secu.rt sur NS1 et vérifié les adresses visibles : ns1 172.25.0.74, ns2 172.25.0.75, client 172.25.0.63 et mail 172.25.0.77. Les lignes exactes de named.conf.local, le SOA complet et les alias ne sont pas lisibles dans les captures conservées.

A.3 – Tests des enregistrements

La zone a été contrôlée avec named-checkzone et le service a été redémarré. Les commandes dig individuelles et leurs réponses complètes ne sont pas récupérables ; nous ne les inventons pas.

B) Création d'une zone inverse

Le retour de `named-checkconf -z` montre aussi le chargement de la zone inverse `0.25.172.in-addr.arpa`. Les détails des enregistrements PTR et les résultats de chaque requête inverse ne sont pas lisibles dans les captures récupérées.

B.1–2 – Zone inverse et tests PTR

named-checkconf -z confirme que la zone inverse 0.25.172.in-addr.arpa est chargée. Les lignes PTR et les résultats des requêtes inverses ne sont pas assez lisibles pour être retranscrits.

V – Configurez un serveur secondaire et mettez en place un transfert sécurisé

1 – Quel est l'intérêt d'avoir un serveur DNS secondaire ?

Un serveur secondaire conserve une copie de la zone. Il peut répondre si le primaire est indisponible et répartit les requêtes.

2 – Installation de BIND9 sur NS2

La conversation confirme que NS2 a été configuré comme serveur secondaire ; la commande d'installation exacte n'est pas lisible dans les éléments récupérés.

3 – Déclaration de la zone esclave

Nous avons configuré NS2 pour la zone secu.rt et la conversation confirme que le transfert fonctionnait. Les lignes exactes de déclaration ne sont pas lisibles.

4 – Vérification du transfert avec Wireshark

Le transfert a été testé et son fonctionnement est confirmé dans la conversation, mais le détail des paquets n'est pas récupérable.

5 – Tests des enregistrements sur NS2

Des tests avec dig à destination de NS2 ont été réalisés et indiqués comme fonctionnels ; les commandes et réponses exactes ne sont pas lisibles.

6 – Qu'apporte une clé TSIG au niveau sécurité ?

La clé TSIG partagée authentifie les échanges et protège leur intégrité, notamment pendant un transfert de zone. Elle ne chiffre pas les données.

7 – Algorithmes supportés par tsig-keygen

La commande du TP utilise HMAC-SHA512. Les algorithmes disponibles dépendent de la version de BIND ; la version documentée propose HMAC-MD5, SHA1, SHA224, SHA256, SHA384 et SHA512. Parmi ceux-ci, HMAC-SHA512 est le plus robuste.

```
tsig-keygen -a HMAC-SHA512
```

8 – Configuration de TSIG sur les deux serveurs

La conversation confirme la configuration de la clé TSIG sur NS1 et NS2. Nous n'intégrons pas la capture de configuration, car elle affiche le secret partagé.

9 – Test du transfert sécurisé

Le transfert sécurisé a été testé et déclaré fonctionnel ; les commandes de redémarrage et l'analyse détaillée de Wireshark ne sont pas lisibles.

10 – Tests des enregistrements sur NS2

La conversation indique que les tests DNS sur NS2 fonctionnaient, mais les réponses exactes ne peuvent pas être confirmées à partir des captures conservées.

11 – Bascule du client après arrêt de NS1

La conversation ne permet pas de confirmer que NS1 a été arrêté et que le client a alors résolu les noms via NS2. Nous ne présentons donc pas cette bascule comme réussie.

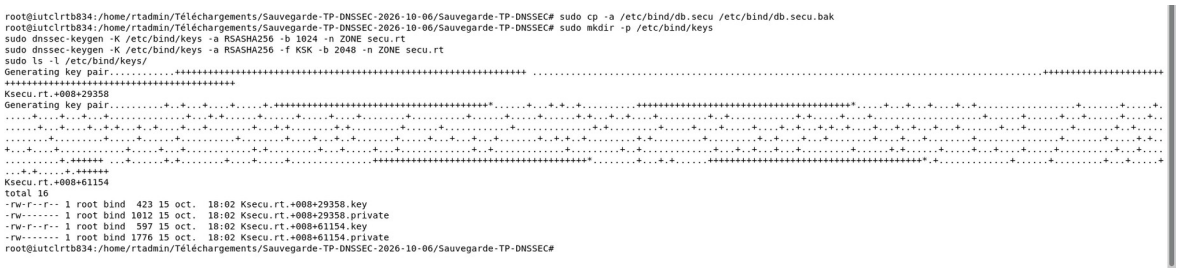
VII – Signature de zone

1 – Génération des clés ZSK et KSK

Nous avons sauvegardé le fichier de zone, créé le dossier des clés, puis généré une paire ZSK et une paire KSK en RSASHA256. La sortie donne le tag **29358** pour la ZSK et **61154** pour la KSK.

```
sudo cp -a /etc/bind/db.secu /etc/bind/db.secu.bak
sudo mkdir -p /etc/bind/keys
sudo dnssec-keygen -K /etc/bind/keys -a RSASHA256 -b 1024 -n ZONE secu.rt
sudo dnssec-keygen -K /etc/bind/keys -a RSASHA256 -f KSK -b 2048 -n ZONE secu.rt
sudo ls -l /etc/bind/keys/
```

L'option -r /dev/urandom, proposée par le sujet si la génération est trop lente, fournit une source d'aléa au programme.

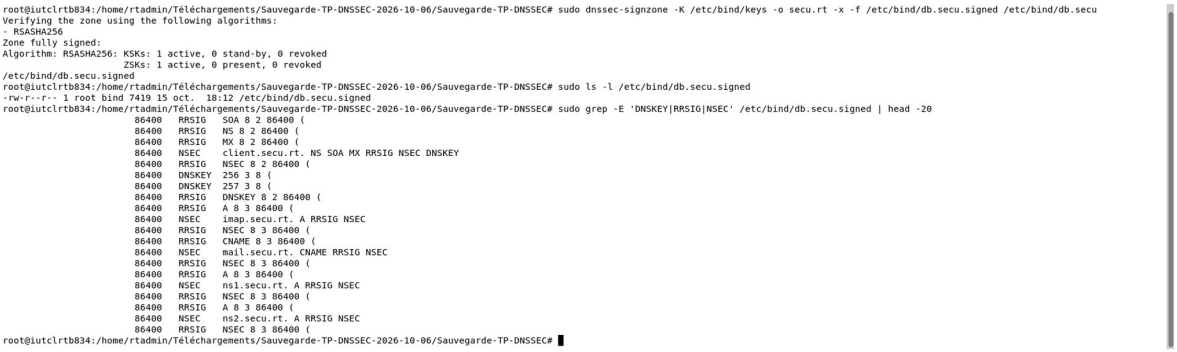


Capture 4 — Génération des clés et tags affichés dans le terminal.

2–3 – Ajout des clés et signature de la zone

Nous avons signé la zone dans un fichier distinct. La sortie confirme une zone complètement signée, avec une KSK et une ZSK actives.

```
sudo dnssec-signzone -K /etc/bind/keys -o secu.rt -x -f /etc/bind/db.secu.signed /etc/bind/db.secu
sudo ls -l /etc/bind/db.secu.signed
sudo grep -E 'DNSKEY|RRSIG|NSEC' /etc/bind/db.secu.signed | head -20
```



Capture 5 — Signature réussie et enregistrements DNSSEC présents.

4 – Vérification des signatures avec dig

```
dig @172.25.0.34 client.secu.rt A +dnssec +multi
```

La réponse A contenait un enregistrement RRSIG, ce qui confirme la présence d'une signature. Ce test correspond à une configuration intermédiaire (172.25.0.34/172.25.0.35), distincte des adresses finales.

5–6 – Signature NSEC3 et vérification

Nous avons ensuite signé la zone avec NSEC3, puis vérifié la zone et la configuration BIND9. named-checkzone renvoie **OK**, et named-checkconf -z indique que la zone directe signée ainsi que la zone inverse sont chargées.

```
sudo dnssec-signzone -K /etc/bind/keys -3 A1B2C3D4 -o secu.rt -x -f /etc/bind/db.secu.nsec3 /etc/bind/db.secu
sudo named-checkzone secu.rt /etc/bind/db.secu.nsec3
sudo named-checkconf -z
sudo systemctl restart bind9
```

6 – Vérification avec l'option +multi

L'option +multi présente les enregistrements sur plusieurs lignes, ce qui facilite la lecture de champs longs comme DNSKEY ou RRSIG. Elle ne modifie pas les données retournées.

VIII – Relais de zone

1 – Configuration du relais

Nous avons configuré NS2 (172.25.0.75) en relais vers NS1 (172.25.0.74) pour la zone secu.rt. Le relais doit disposer de la clé publique KSK comme ancre de confiance afin de valider les réponses signées.

```
trust-anchors {
  "secu.rt." static-key 257 3 8 "CLE_PUBLIQUE_KSK";
};
```

La clé utilisée est la KSK de secu.rt, tag **61154**. La valeur complète de la clé n'est pas reproduite dans le compte rendu.

2 – Requêtes DNSSEC et résultats

Avant l'ajout de l'ancre de confiance, les requêtes DNSKEY sur le relais renvoyaient SERVFAIL. Après l'ajout de la bonne KSK, la résolution positive a renvoyé **172.25.0.63** avec le flag ad : les données étaient alors considérées comme authentifiées par DNSSEC.

```
dig @127.0.0.1 secu.rt DNSKEY +dnssec +multi
dig @127.0.0.1 secu.rt DNSKEY +dnssec +multi +ad
```

```
root@iutclrtc715:/home/rtadmin# sudo nano /etc/bind/named.conf.local
root@iutclrtc715:/home/rtadmin# sudo named-checkconf
sudo systemctl restart bind9
root@iutclrtc715:/home/rtadmin# dig @127.0.0.1 secu.rt DNSKEY +dnssec +multi

; <<>> DiG 9.20.29-1~deb13u1-Debian <<>> @127.0.0.1 secu.rt DNSKEY +dnssec +multi
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 41734
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: e69079d6fafa1f31010000006ad76f413067bef22e03de14 (good)
;; QUESTION SECTION:
;secu.rt.               IN DNSKEY

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Tue Oct 20 15:40:17 CEST 2026
;; MSG SIZE  rcvd: 64

root@iutclrtc715:/home/rtadmin# dig @127.0.0.1 secu.rt DNSKEY +dnssec +multi +ad

; <<>> DiG 9.20.29-1~deb13u1-Debian <<>> @127.0.0.1 secu.rt DNSKEY +dnssec +multi +ad
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 40147
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 8e640733d3f0d137010000006ad76f4759aab14a2542e3d4 (good)
;; QUESTION SECTION:
;secu.rt.               IN DNSKEY

;; Query time: 0 msec
;; SERVER: 127.0.0.1#53(127.0.0.1) (UDP)
;; WHEN: Tue Oct 20 15:40:23 CEST 2026
;; MSG SIZE  rcvd: 64
```

Capture 6 — Réponse SERVFAIL avant la correction de l'ancre de confiance.

Dans les flags de dig, qr indique une réponse, rd une récursivité demandée, ra une récursivité disponible, aa une réponse autoritaire et ad des données authentifiées par DNSSEC. L'option +ad positionne le bit AD dans la requête ; elle ne force pas le serveur à valider les signatures.

Le test négatif sur absent.secu.rt renvoie NXDOMAIN avec des preuves NSEC3 signées. Dans le test de la conversation, le flag ad était absent : ce point n'est donc pas validé.

2 – Interprétation des flags et rôle de +ad

Les flags observés sont décrits ci-dessus ; ad indique que le résolveur considère les données comme authentifiées. L'option +ad demande le bit AD dans la requête, mais ne remplace pas la validation DNSSEC par le serveur.

Relais, erreur initiale et ancre de confiance

Le relais a d'abord répondu SERVFAIL, puis la résolution positive a fonctionné après ajout de la bonne clé KSK comme trust-anchor ; la réponse contenait 172.25.0.63 et le flag ad. Cela montre que le relais avait besoin de la clé publique de confiance pour valider la chaîne de signatures.

Note sur les captures

Les captures retenues proviennent du partage de conversation et illustrent les étapes de vérification et de signature. La capture de configuration TSIG a été écartée car elle exposait le secret partagé. Les valeurs non lisibles dans les captures n'ont pas été reconstituées.